

I.N.A.Y.A.T. Technical Architecture & System Implementation

This technical report details the architecture, implementation steps, and operational flow of the MCA 4th Semester Final Project: I.N.A.Y.A.T. (Intelligent Neural Architecture for Yielding Agentic Thinking). It describes what was built, how it was built, how the system operates, and the gap analysis of the project.

1. WHAT WE CREATED (PROJECT BLUEPRINT)

We established a complete production-grade workspace structure with all 21 scaffolding and functional files:

- * **app.py:** Streamlit UI interface, state manager, and chat layout.
- * **core/llm_setup.py:** Google GenAI LlamaIndex settings, LLM initialization, and embed model configuration.
- * **core/memory.py:** Mem0 client integration, user fact extraction, and memory cleaner.
- * **core/graph_store.py:** Neo4j database connection pools and Cypher query execution wrapper.
- * **core/agent.py:** LlamaIndex PropertyGraphIndex builder and hybrid vector-keyword retrieval engine.
- * **core/resilience.py:** Self-healing Tenacity retry decorators and custom Circuit Breaker classes.
- * **core/health.py:** HealthMonitor class providing status checks for Gemini, Mem0, and Neo4j.
- * **core/startup.py:** Environment validator and health sweep at boot-up.
- * **core/logging_config.py:** Rotating file (inayat_debug.log) and console logger.
- * **tests/smoke_test.py:** A comprehensive suite of 17 tests validating core systems.
- * **warmup.py:** Keep-alive script to verify APIs and prevent Neo4j AuraDB free tiers from pausing.

2. HOW WE CREATED IT (PROCESS & WORKFLOW)

- * **Python 3.14 Environment:** Configured a local virtual environment (.venv) under Windows. Because Python 3.14 does not have legacy wheels on PyPI, we resolved dependency conflicts by fetching pre-compiled binary packages for numpy (2.4.6) and pillow (12.2.0), avoiding C++ compiler errors on Windows.
- * **Google GenAI Integration:** Replaced deprecated LlamaIndex gemini packages with the modern 'llama-index-llms-google-genai' and 'llama-index-embeddings-google-genai' libraries, resolving Pillow constraints.
- * **SDK v2 Parameter Mapping:** Resolved Mem0 SDK version 2 API changes by passing parameters via the filters={'user_id': ...} dictionary and parsing data using the paginated raw.get('results', []) format.

3. WHAT WE IMPLEMENTED & SYSTEM WORKFLOW

The system operates in a circular feedback loop ensuring personalization, contextual RAG search, and memory updates:

- * **Step 1: User Input:** The user enters a question in the Streamlit UI. The system validates the session and user identity.
- * **Step 2: Memory Retrieval:** The client queries Mem0 Cloud using filters={'user_id': user_id}. If memories exist, they are compiled into a bulleted list of facts.
- * **Step 3: Property Graph Retrieval (RAG):** LlamaIndex queries the Neo4j AuraDB graph store using a PropertyGraphIndex (hybrid vector similarity + keyword matching) over your uploaded PDFs to extract relevant context.
- * **Step 4: LLM Synthesis:** The prompt is augmented with both long-term facts (from Mem0) and document context (from Neo4j). Gemini 1.5 Flash generates a context-aware answer.
- * **Step 5: Memory Update:** The new interaction is asynchronously pushed to Mem0. Mem0 extracts new facts, resolves contradictions, and updates the profile.
- * **Step 6: Graceful Degradation:** If Mem0 or Neo4j fails, the circuit breaker opens. The system automatically bypasses the failing service (e.g. falling back to pure Gemini chat without memory or documents) without crashing.

4. WHAT IS MISSING (GAP ANALYSIS)

- * **Graphical Visualizations:** While the Neo4j backend holds nodes and edges, the Streamlit page only shows text. To show a live visual canvas of the knowledge graph inside the UI, we would need to integrate a library like streamlit-agraph.
- * **User Cookie Persistence:** The user profile defaults to a text box which resets on page reload. A cookie manager is missing.

5. WORK DONE VS. WORK LEFT

- * **Work Done:** Full package builds, LlamaIndex Google GenAI migration, Mem0 v2 filter correction, circuit breaker sentinel pattern, ASCII logging fix, 17 smoke tests passing, and git code commit.
- * **Work Left:** Renaming .env.example to .env and entering real keys, uploading PDFs to data/documents/, and booting up the app.